

CRUD REPOSITORY

Interface for generic CRUD operations on a repository for a specific type. Methods exposed through this interface allow entities to participate in lifecycle events, and optimistic locking if applicable except for some bulk operation methods. Therefore, modules may choose to load an entity before deleting or updating it in order to facilitate events, and any modifying method call may trigger an exception due to failure of optimistic locking.

Modifier and Type	Method	Description
long	<code>count()</code>	Returns the number of entities available.
void	<code>delete(T entity)</code>	Deletes a given entity.
void	<code>deleteAll()</code>	Deletes all entities managed by the repository.
void	<code>deleteAll(Iterable[Ⓜ]<? extends T> entities)</code>	Deletes the given entities.
void	<code>deleteAllById(Iterable[Ⓜ]<? extends ID> ids)</code>	Deletes all instances of the type T with the given IDs.
void	<code>deleteById(ID id)</code>	Deletes the entity with the given id.
boolean	<code>existsById(ID id)</code>	Returns whether an entity with the given id exists.
<code>Iterable[Ⓜ]<T></code>	<code>findAll()</code>	Returns all instances of the type.
<code>Iterable[Ⓜ]<T></code>	<code>findAllById(Iterable[Ⓜ]<ID> ids)</code>	Returns all instances of the type T with the given IDs.
<code>Optional[Ⓜ]<T></code>	<code>findById(ID id)</code>	Retrieves an entity by its id.
<code><S extends T> S</code>	<code>save(S entity)</code>	Saves a given entity.
<code><S extends T> Iterable[Ⓜ]<S></code>	<code>saveAll(Iterable[Ⓜ]<S> entities)</code>	Saves all given entities.

Example:

Create a Spring Starter Project with enabling dependencies Lombok, MySQL Driver and Spring Data JPA. Create a database and mention its name in application.properties file

```
application.properties ×
1 spring.application.name=CRUD-Repository
2
3 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
4 spring.datasource.url=jdbc:mysql://localhost:3306/SpringCrud1
5 #SpringCrud1 is the database name
6 spring.datasource.username=root
7 spring.datasource.password=tiger
8
9 spring.jpa.show-sql=true
10 #prints the query on the console -- shows all the queries that are run on the console window
11
12 spring.jpa.hibernate.ddl-auto=update
```

Create a base package and sub packages entity, repo and runner inside which the respective classes will be created. (Same procedure for the following examples in repository)

TaskManager.java (Entity class)

```
TaskManager.java X
1 package com.example.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.Table;
6 import lombok.AllArgsConstructor;
7 import lombok.Data;
8 import lombok.NoArgsConstructor;
9
10 @Data
11 @NoArgsConstructor
12 @AllArgsConstructor
13 @Entity
14 @Table(name="Tasktab")
15 public class TaskManager {
16
17     @Id
18     private Integer taskId;
19     private String taskName;
20     private String dueDate;
21 }
22
```

TaskManagerRepository.java (Interface)

```
TaskManagerRepository.java X
1 package com.example.repo;
2
3 import org.springframework.data.repository.CrudRepository;
4
5 import com.example.entity.TaskManager;
6
7 public interface TaskManagerRepository extends CrudRepository<TaskManager, Integer>{
8
9 }
10
```

TaskRunner.java (Class which implements CommandLineRunner)

1. save()

```
TaskRunner.java X
15
16 @Component
17 public class TaskRunner implements CommandLineRunner{
18
19     @Autowired
20     private TaskManagerRepository trepo;
21
22     @Override
23     public void run(String... args) throws Exception {
24
25         System.out.println("1. save() method");
26         //Saves a given entity. Returns: the saved entity; will never be null.
27         //Use the returned instance for further operations as the save operation
28         //might have changed the entity instance completely.
29
30         trepo.save(new TaskManager(10,"Prepare Presentation","1/10/2025"));
31         trepo.save(new TaskManager(11,"Prepare Spring Notes","25/9/2025"));
32         trepo.save(new TaskManager(12,"Prepare for Spring Mock","22/9/2025"));
33         trepo.save(new TaskManager(13,"Revise Basics","30/9/2025"));
34         trepo.save(new TaskManager(14,"Prepare for Interview","30/9/2025"));
35         trepo.save(new TaskManager(15,"Develop Spring Project","20/11/2025"));
36         System.out.println("-----");
37     }
38 }
```

2. saveAll()

```
TaskRunner.java X
37
38 System.out.println("2. saveAll() method");
39 //Saves all given entities.
40 //Returns : the saved entities; will never be null.
41 //The returned Iterable will have the same size as the Iterable passed as an argument.
42 trepo.saveAll(Arrays.asList(
43     new TaskManager(16,"Attend Interview","30/9/2025"),
44     new TaskManager(17,"Prepare frontend technologies","10/11/2025"),
45     new TaskManager(18,"Revise Everything","1/10/2025")
46 ));
47
48 System.out.println("-----");
49
```

3. findAll()

```
TaskRunner.java X
49
50 System.out.println("3. findAll() method");
51 //Returns all instances of the type.
52
53 Iterable<TaskManager> displaytasks = trepo.findAll();
54 displaytasks.forEach(System.out::println);
55
56 //or
57 System.out.println();
58 trepo.findAll().forEach(System.out::println);
59
60 System.out.println("-----");
61
```

4.findById()

```
TaskRunner.java X
61
62
63 System.out.println("4. findById() method");
64
65 // Retrieves an entity by its id.
66 //
67 // Returns : the entity with the given id or Optional#empty() if none found.
68
69 //check for both conditions id present and id not present
70 trepo.findById(10).ifPresentOrElse(System.out::println, ()->{System.out.println("TASK DATA FOUND");});
71
72 //or
73
74 Optional<TaskManager> task = trepo.findById(29);
75
76 if(task.isPresent()) {
77     System.out.println(task.get());
78 }else {
79     System.out.println("TASK DATA FOUND");
80 }
81
82 System.out.println("-----");
83
```

5. existsById()

```
TaskRunner.java X
84
85 System.out.println("5. existsById() method");
86
87 // Returns whether an entity with the given id exists.
88 // Returns : true if an entity with the given id exists, false otherwise.
89
90
91 boolean isTaskPresent = trepo.existsById(10);
92 System.out.println(isTaskPresent);
93
94 //or
95
96 System.out.println(trepo.existsById(39));
97
98 System.out.println("-----");
99
```

6. count()

```
TaskRunner.java ×
100
101     System.out.println("6. count() method");
102
103     //Returns the number of entities available.
104
105     long count = trepo.count();
106     System.out.println(count);
107     //or
108     System.out.println(trepo.count());
109
110     System.out.println("-----");
111
```

7. findAllById()

```
TaskRunner.java ×
112
113     System.out.println("7. findAllById() method");
114
115     //Returns all instances of the type T with the given IDs.
116
117     Iterable<TaskManager> tasklist = trepo.findAllById(Arrays.asList(11,12,13,44,76));
118     tasklist.forEach(System.out::println);
119     //or
120     System.out.println();
121     for(TaskManager task1 : tasklist) {
122         System.out.println(task1);
123     }
124
125     System.out.println("-----");
126
```

8. deleteById()

```
TaskRunner.java ×
126
127
128     System.out.println("8. deleteById() method");
129     //Deletes the entity with the given id.
130     //If the entity is not found it is silently ignored.
131
132     if(trepo.existsById(9)) {
133         trepo.deleteById(9);
134         System.out.println("TASK DELETED");
135     } else {
136         System.out.println("TASK NOT FOUND");
137     }
138
139     //or
140
141     trepo.findById(18).ifPresentOrElse(
142         k->{
143             trepo.deleteById(k.getTaskId());
144             System.out.println("DELETED TASK : "+k);
145         },
146         ()->{System.out.println("TASK NOT FOUND");}
147     );
148
149     System.out.println("-----");
150
```

9. delete()

```
150
151     System.out.println("9. delete() method");
152     //Deletes a given entity.
153
154     trepo.findById(17).ifPresent(
155         k->{
156             trepo.delete(k);
157             System.out.println("RECORD DELETED "+ k);
158         }
159     );
160
161     System.out.println("-----");
162
```

10. deleteAllById()

```
163      System.out.println("10. deleteAllById()");
164      //Deletes all instances of the type T with the given IDs.
165      //Entities that aren't found are silently ignored.
166
167      List<Integer> Ids = Arrays.asList(13,14);
168      trepo.deleteAllById(Ids);
169      System.out.println("ID'S OF RECORDS DELETED "+Ids);
170      //or
171      trepo.deleteAllById(Stream.of(15,16).collect(Collectors.toList()));
172
173      System.out.println("-----");
174
175
```

11. deleteAll()

```
176      System.out.println("11. deleteAll() method");
177      //Deletes all entities managed by the repository.
178
179      trepo.deleteAll();
180
181
```

QUERY METHODS

Standard CRUD functionality repositories usually have queries on the underlying datastore. With Spring Data, declaring those queries becomes a four-step process:

1. Declare an interface extending Repository or one of its sub interfaces and type it to the domain class and ID type that it should handle, as shown in the following

example:

```
interface PersonRepository extends Repository<Person, Long> { ... }
```

2. Declare query methods on the interface.

```
interface PersonRepository extends Repository<Person, Long> {
    List<Person> findByLastname(String lastname);
}
```

3. Inject the repository instance and use it, as shown in the following example:

```
class PersonRunner implements CommandLineRunner{
    @Autowired
    PersonRepository prepo;

    public void run(String... args) throws Exception {
        ....
        prepo.findByLastname("Sharma").forEach(System.out::println);
    }
}
```

Keyword	Sample	JPQL snippet
Distinct	findDistinctByLastnameAndFirstname	select distinct ... where x.lastname = ?1 and x.firstname = ?2
And	findByLastnameAndFirstname	... where x.lastname = ?1 and x.firstname = ?2
Or	findByLastnameOrFirstname	... where x.lastname = ?1 or x.firstname = ?2
Is, Equals	findByFirstname, findByFirstnames, findByFirstnameEquals	... where x.firstname = ?1 (or ... where x.firstname IS NULL if the argument is null)
Between	findByStartDateBetween	... where x.startDate between ?1 and ?2
LessThan	findByAgeLessThan	... where x.age < ?1
LessThanEqual	findByAgeLessThanEqual	... where x.age <= ?1
GreaterThan	findByAgeGreaterThan	... where x.age > ?1
GreaterThanEqual	findByAgeGreaterThanEqual	... where x.age >= ?1
After	findByStartDateAfter	... where x.startDate > ?1
Before	findByStartDateBefore	... where x.startDate < ?1
IsNull, Null	findByAge(Is)Null	... where x.age is null
IsNotNull,NotNull	findByAge(Is)NotNull	... where x.age is not null
Like	findByFirstnameLike	... where x.firstname like ?1
NotLike	findByFirstnameNotLike	... where x.firstname not like ?1
StartingWith	findByFirstnameStartingWith	... where x.firstname like ?1 (parameter bound with appended %)
EndingWith	findByFirstnameEndingWith	... where x.firstname like ?1 (parameter bound with prepended %)
Containing	findByFirstnameContaining	... where x.firstname like ?1 (parameter bound wrapped in %)
OrderBy	findByAgeOrderByLastnameDesc	... where x.age = ?1 order by x.lastname desc
Not	findByLastnameNot	... where x.lastname <> ?1 (or ... where x.lastname IS NOT NULL if the argument is null)
In	findByAgeIn(Collection<Age> ages)	... where x.age in ?1
NotIn	findByAgeNotIn(Collection<Age> ages)	... where x.age not in ?1
True	findByActiveTrue()	... where x.active = true
False	findByActiveFalse()	... where x.active = false
IgnoreCase	findByFirstnameIgnoreCase	... where UPPER(x.firstname) = UPPER(?1)

Example:

Events.java (Entity class)

```
Events.java X
1 package com.example.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.Id;
6 import jakarta.persistence.Table;
7 import lombok.AllArgsConstructor;
8 import lombok.Data;
9 import lombok.NoArgsConstructor;
10
11 @Data
12 @Entity
13 @AllArgsConstructor
14 @NoArgsConstructor
15 @Table(name = "Eventtab")
16 public class Events {
17
18     @Id
19     @Column(name = "event_id")
20     private Integer eventId;
21
22     @Column(name = "event_name")
23     private String eventName;
24
25     @Column(name = "location")
26     private String location;
27
28     @Column(name = "event_date")
29     private String eventDate;
30
31 }
32
```

EventsRepository.java (Interface)

```
EventsRepository.java X EventsRunner.java
1 package com.example.repo;
2
3 import org.springframework.data.jpa.repository.Modifying;
4 import org.springframework.data.jpa.repository.Query;
5 import org.springframework.data.repository.CrudRepository;
6 import org.springframework.data.repository.query.Param;
7
8 import com.example.entity.Events;
9
10 import jakarta.transaction.Transactional;
11
12 import java.util.Collection;
13 import java.util.List;
14
15
16 public interface EventsRepository extends CrudRepository<Events, Integer>{
17
```

Query methods defined inside the repository interface are as follows :

1. searchBy methods

```
97
98 //select * from events where event_id in (?, ?, ?, ?, ...)
99 List<Events> searchByEventIdIn(Collection<Integer> eventId);
100
101 //select * from events where event_name=?
102 List<Events> searchByEventName(String eventName);
103
104 //select * from events where event_date=?
105 List<Events> searchByEventDate(String eventDate);
106
107 //select * from events where location=?
108 List<Events> searchByLocation(String location);
109
```

2. findBy methods

```
16 public interface EventsRepository extends CrudRepository<Events, Integer>{
17
18     //select * from events where event_date=?
19     List<Events> findByEventDate(String eventDate);
20
21     //select * from events where event_name=?
22     List<Events> findByEventName(String eventName);
23
24     //select * from events where location=?
25     List<Events> findByLocation(String location);
26
27     //select * from events where event_name like=? order by event_id
28     List<Events> findByEventNameLikeOrderByEventId(String eventName);
29
30     //select * from events where event_name=? or location=?
31     List<Events> findByEventNameOrLocation(String eventName, String location);
32
33     //select * from events where event_id=? or event_name=?
34     List<Events> findByIdOrEventName(Integer eventId, String eventName);
35
36     //select * from events where location=? or event_date=?
37     List<Events> findByLocationOrEventDate(String location, String eventDate);
38
39     //select * from events where event_date=? order by event_id
40     List<Events> findByEventDateOrderByEventId(String eventDate);
41
42     //select * from events where lower(event_name) = lower(?)
43     List<Events> findByAllIgnoringCaseEventName(String eventName);
44 }
```

3. getBy methods

```
48
49 //select * from events where event_id between ? and ?
50 List<Events> getByEventIdBetween(Integer e1,Integer e2);
51
52 //select * from events where event_name=?
53 List<Events> getByEventName(String eventName);
54
55 //select * from events where event_date=?
56 List<Events> getByEventDate(String eventDate);
57
58 //select * from events where location=?
59 List<Events> getByLocation(String location);
60
61 //select * from events where event_name=? and location=?
62 List<Events> getByEventNameAndLocation(String eventName, String location);
63
64 //select * from events where event_id=?
65 List<Events> getByEventIdGreaterThan(Integer eventId);
66
67 //select * from events where location like concat('%',?)
68 List<Events> getByLocationEndingWith(String s);
69
```

4. removeBy methods

```
71
72 //delete from events where event_id=?
73 void removeById(Integer eventId);
74
75 //delete from events where event_name=?
76 void removeByName(String eventName);
77
78 //delete from events where location=?
79 void removeByLocation(String location);
80
81 //delete from events where event_date=?
82 void removeByEventDate(String eventDate);
83
84 //delete from events where lower(event_name) = lower(?)
85 void removeByAllIgnoreCaseEventName(String eventName);
86
87 //delete from events where lower(location) = lower(?)
88 void removeByAllIgnoreCaseLocation(String location);
89
90 //delete from events where event_name=? or location=?
91 void removeByEventNameOrLocation(String eventName, String location);
92
93 //delete from events where event_name like concat('%',?)
94 void removeByEventNameStartingWith(String s);
95
```


NATIVE SQL QUERIES

- Using the `@Query` or `@NativeQuery` annotation allows running native queries
- Note: The `@NativeQuery` annotation is mostly a composed annotation for `@Query(nativeQuery=true)` but it also provides additional attributes such as `sqlResultSetMapping` to leverage JPA's `@SqlResultSetMapping(...)`.

```
EventsRepository.java  EventsRunner.java
112 //-----NATIVE SQL QUERIES-----
113
114 @Query(value="Select * from Events where event_name like concat('%',?1 suffix )",nativeQuery = true)
115 List<Events> findByEventNameEndingWith(String suffix);
116
117 @Query(value="Select * from Events where event_name like concat(?1 prefix ,'%')",nativeQuery = true)
118 List<Events> findByEventNameStartingWith(String prefix);
119
120 @Modifying@Transactional
121 @Query(value="Delete from Events where event_id = :id",nativeQuery = true)
122 void deleteById(@Param("id") Integer id);
123
124 @Modifying@Transactional
125 @Query(value="Delete from Events where event_name=?1 name ",nativeQuery = true)
126 void deleteByName(String name);
127
128 @Modifying@Transactional
129 @Query(value="Delete from Events where event_date=?1 eventDate ",nativeQuery = true)
130 void deleteByEventDate(String eventDate);
131
132 @Modifying@Transactional
133 @Query(value="Delete from Events where location=:loc",nativeQuery = true)
134 void deleteByLocation(@Param("loc") String location);
135
136 @Modifying@Transactional
137 @Query(value="Update Events Set event_name=?2 name where event_id=?1 id ",nativeQuery = true)
138 void updateEventName(Integer id,String name);
139
140 @Modifying@Transactional
141 @Query(value="Update Events Set event_date=?2 date where event_id=?1 id ",nativeQuery = true)
142 void updateEventDate(Integer id, String date);
143
144 @Modifying@Transactional
145 @Query(value="Update Events Set location=?2 location where event_id=?1 id ",nativeQuery = true)
146 void updateLocation(Integer id, String location);
147
148 @Query(value="Select * from Events where location=?1 location and event_date=?2 eventDate ",nativeQuery = true)
149 List<Events> findByLocationAndEventDate(String location,String eventDate);
150
151
```

HQL QUERIES

```
152 //-----HQL QUERIES-----
153
154 @Query("Select count(e) from Events e")
155 Integer findCountOfEvents();
156
157 @Query("Select Count(e) from Events e where e.location=:location")
158 Integer findEventsByLocation(String location);
159
160 @Query("Select Count(e) from Events e where e.eventDate= :eventDate")
161 Integer findEventsByEventDate(String eventDate);
162
163 @Query("Select Count(e) from Events e where e.eventName= :eventName")
164 Integer findEventsByEventName(String eventName);
165
166 @Query("From Events e where e.location not like :loc ")
167 List<Events> findEventsNotLocation(@Param("loc") String location);
168
169 @Query("From Events e where lower(e.eventName) = lower(:eventName)")
170 List<Events> findEventsByEventNameIgnoringCase(String eventName);
171
172 @Query("From Events e order By e.eventName")
173 List<Events> findEventsOrderByEventName();
174
175 @Query("From Events e order By e.eventDate")
176 List<Events> findEventsOrderByEventDate();
177
178 }
179
```

EventsRunner.java

```
EventsRunner.java X
1 package com.example.runner;
2
3 import java.util.Arrays;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 import com.example.entity.Events;
10 import com.example.repo.EventsRepository;
11
12 import jakarta.transaction.Transactional;
13
14 @Component
15 @Transactional
16 public class EventsRunner implements CommandLineRunner{
17
18     @Autowired
19     private EventsRepository erepo;
20
21     @Override
22     public void run(String... args) throws Exception {
23         // TODO Auto-generated method stub
24         System.out.println();
25         erepo.saveAll(
26             Arrays.asList(new Events(121,"Technovate","Bengaluru","25/10/2025"),
27                 new Events(122,"Hackathon","Noida","26/10/2025"),
28                 new Events(123,"TechCoding","Gurugram","27/10/2025"),
29                 new Events(124,"Algotest","Mumbai","28/10/2025"),
30                 new Events(125,"Code 360","Delhi","29/10/2025"),
31                 new Events(126,"CodeSprint","Bengaluru","30/10/2025"),
32                 new Events(127,"Hack n Snack","Bengaluru","31/10/2025"),
33                 new Events(128,"BugBusters","Mumbai","1/11/2025"),
34                 new Events(129,"ScreenStorm","Hyderabad","2/11/2025"),
35                 new Events(130,"FutureHub","Bengaluru","3/11/2025")
36             ));
37
38
39         System.out.println("-----");
```

//execution of findBy methods

```
EventsRunner.java X
40
41     erepo.findByEventDate("2/11/2025").forEach(System.out::println);
42
43     erepo.findByEventName("Technovate").forEach(System.out::println);
44
45     erepo.findByLocation("Bengaluru").forEach(System.out::println);
46
47     erepo.findByEventNameLikeOrderByEventId("Code").forEach(System.out::println);
48
49     erepo.findByEventNameOrLocation("Code 360","Bengaluru").forEach(System.out::println);
50
51     erepo.findByEventIdOrEventName(120,"CodeSpirit").forEach(System.out::println);
52
53     erepo.findByLocationOrEventDate("Bengaluru","2/11/2025").forEach(System.out::println);
54
55     erepo.findByEventDateOrderByEventId("1/11/2025").forEach(System.out::println);
56
57     erepo.findByAllIgnoringCaseEventName("Hack n Snack").forEach(System.out::println);
58
59     System.out.println("-----");
60
```

//execution of getBy methods

```
EventsRunner.java ×
62
63     erepo.getByEventIdBetween(121,125).forEach(System.out::println);
64
65     erepo.getByEventName("BugBusters").forEach(System.out::println);
66
67     erepo.getByEventDate("29/10/2025").forEach(System.out::println);
68
69     erepo.getByLocation("Noida").forEach(System.out::println);
70
71     erepo.getByEventNameAndLocation("FutureHub", "Hyderabad").forEach(System.out::println);
72
73     erepo.getByEventIdGreaterThan(122).forEach(System.out::println);
74
75     erepo.getByLocationEndingWith("i").forEach(System.out::println);
76
77     System.out.println("-----");
78
```

//execution of removeBy methods

```
EventsRunner.java ×
80
81     erepo.removeById(122);
82
83     erepo.removeByName("Algotest");
84
85     erepo.removeByLocation("Hyderabad");
86
87     erepo.removeByEventDate("28/10/2025");
88
89     erepo.removeByAllIgnoreCaseEventName("TechCoding");
90
91     erepo.removeByAllIgnoreCaseLocation("Gurugram");
92
93     erepo.removeByEventNameOrLocation("Hackathon", "Mumbai");
94
95     erepo.removeByEventNameStartingWith("ScreenStorm");
96
97     erepo.findAll().forEach(System.out::println);
98
99     System.out.println("-----");
100
```

//execution of searchBy methods

```
EventsRunner.java ×
102
103     erepo.searchByEventIdIn(Arrays.asList(121,126,127)).forEach(System.out::println);
104
105     erepo.searchByEventName("Algotest").forEach(System.out::println);
106
107     erepo.searchByLocation("Noida").forEach(System.out::println);
108
109     erepo.searchByEventDate("30/09/2025").forEach(System.out::println);
110
111     System.out.println("-----");
112
```

//execution of native queries

```
EventsRunner.java ×
114     System.out.println("Native queries-----");
115     |
116     erepo.findByEventNameEndingWith("t").forEach(System.out::println);
117
118     erepo.findByEventNameStartingWith("C").forEach(System.out::println);
119
120     erepo.deleteByEventId(22);
121
122     erepo.deleteByEventName("Hackathon");
123
124     erepo.deleteByEventDate("20/10/2025");
125
126     erepo.deleteBylocation("Bengaluru");
127
128     erepo.updateEventName(126, "CodeSpirit");
129
130     erepo.updateEventDate(129, "19/10/2025");
131
132     erepo.updateLocation(130, "Noida");
133
134     erepo.findByLocationAndEventDate("Bengaluru", "23/10/2025");
135
136
```

//execution of HQL queries

```
138
139     System.out.println("HQL queries-----");
140
141     System.out.println(erepo.findCountOfEvents());
142
143     System.out.println(erepo.findEventsByLocation("Bengaluru"));
144
145     System.out.println(erepo.findEventsByEventDate("19/10/2025"));
146
147     System.out.println(erepo.findEventsByEventName("Hackathon"));
148
149     erepo.findEventsNotLocation("Bengaluru").forEach(System.out::println);
150
151     erepo.findEventsByEventNameIgnoringCase("futurehub");
152
153     erepo.findEventsOrderByEventName().forEach(System.out::println);
154
155     erepo.findEventsOrderByEventDate().forEach(System.out::println);
156
157 }
158
159 }
160
```

JPA REPOSITORY

JPQL Queries

Example:

Recipe.java (Entity class)

```
Recipe.java X
1 package com.example.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 @Data
10 @Entity
11 @AllArgsConstructor
12 @NoArgsConstructor
13 public class Recipe {
14
15     @Id
16     private Integer recipeId;
17     private String recipeName;
18     private String cookTime;
19 }
20
```

RecipeRepository.java (Interface)

```
RecipeRepository.java X
1 package com.example.repo;
2
3 import java.util.Collection;
4 import java.util.List;
5
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Modifying;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
11 import com.example.entity.Recipe;
12
13 import jakarta.transaction.Transactional;
14
15 public interface RecipeRepository extends JpaRepository<Recipe, Integer>{
16
17     @Query("Select r From Recipe r where r.cookTime = :time")
18     List<Recipe> getAllRecipesBycookTime(@Param("time") String cookTime);
19
20     @Query("Select r From Recipe r where r.recipeName = :name")
21     List<Recipe> getAllRecipesByName(@Param("name") String recipeName);
22
23     @Transactional
24     @Modifying
25     @Query("Update Recipe r Set r.recipeName =?1 recipeName where r.recipeId=?2 id ")
26     public int updateRecipeName(String recipeName,int id);
27
28 }
```

```

27
28 @Transactional
29 @Modifying
30 @Query("Update Recipe r Set r.cookTime =?1 cookTime where r.recipeId=?2 id ")
31 public int updateCookTime(String cookTime,int id);
32
33 @Modifying
34 @Query("Delete from Recipe r where r.recipeName =?1 recipeName ")
35 public void deleteByRecipeName(String recipeName);
36
37 @Query("Select r From Recipe r where r.recipeName =?1 recipeName or r.cookTime=?2 cookTime ")
38 List<Recipe> searchByRecipeNameOrCookTime(String recipeName,String cookTime);
39
40 @Query("Select r From Recipe r where r.recipeId=?1 recipeId or r.recipeName =?2 recipeName ")
41 List<Recipe> searchByRecipeIdorRecipeName(Integer recipeId,String recipeName);
42
43 @Query("Select r from Recipe r where lower(r.recipeName) = lower(?1 recipeName )")
44 List<Recipe> findByRecipeNameIgnoreCase(String recipeName);
45
46 @Query("Select r from Recipe r where r.recipeName in ?1 recipeName ")
47 List<Recipe> searchByRecipeNameIn(Collection<String> recipeName);
48
49 @Query("Select r from Recipe r where r.recipeName=?1 recipeName or r.cookTime=?2 cookTime order by r.cookTime ASC")
50 List<Recipe> searchByRecipeNameOrCookTimeOrderByCookTime(String recipeName,String cookTime);
51 }

```

RecipeRunner.java (Class which implements CommandLineRunner)

```

RecipeRunner.java X
1 package com.example.runner;
2
3 import java.util.Arrays;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 import com.example.entity.Recipe;
10 import com.example.repo.RecipeRepository;
11
12 import jakarta.transaction.Transactional;
13
14 @Component
15 @Transactional
16 public class RecipeRunner implements CommandLineRunner{
17
18     @Autowired
19     private RecipeRepository repo;
20
21
22     @Override
23     public void run(String... args) throws Exception {
24         repo.saveAll(Arrays.asList(
25             new Recipe(131,"Greek Salad","10mins"),
26             new Recipe(132,"French Toast","15mins"),
27             new Recipe(133,"Banana pancakes","20mins"),
28             new Recipe(134,"Margherita Pizza","30mins"),
29             new Recipe(135,"Vegetable Stir Fry","15mins"),
30             new Recipe(136,"Biryani","75mins"),
31             new Recipe(137,"Lasagna","60mins"),
32             new Recipe(138,"Sushi rolls","50mins"),
33             new Recipe(139,"Moussaka","75mins"),
34             new Recipe(140,"Kimchi Jjigae","40mins")
35         ));
36

```

```

RecipeRunner.java ×
37     System.out.println();
38     repo.getAllRecipesByCookTime("40mins").forEach(System.out::println);
39     System.out.println("-----");
40
41     repo.getAllRecipesByName("Biryani").forEach(System.out::println);
42     System.out.println("-----");
43
44
45     repo.updateRecipeName("Vegetable Biryani",136);
46
47     System.out.println("-----");
48
49
50     repo.updateCookTime("90mins", 138);
51
52
53
54     System.out.println("-----");
55
56     repo.deleteByRecipeName("Lasagna");
57     System.out.println("-----");
58
59     repo.searchByRecipeNameOrCookTime("Sushi rolls", "30mins").forEach(System.out::println);
60     System.out.println("-----");
61
62     repo.searchByRecipeIdOrRecipeName(139, "French Toast").forEach(System.out::println);
63     System.out.println("-----");
64
65     repo.findByRecipeNameIgnoreCase("banana pancakes").forEach(System.out::println);
66     System.out.println("-----");
67
68     repo.searchByRecipeNameIn(Arrays.asList("Greek Salad", "Kimchi Jjigae", "Moussaka", "Choco Chips Cookies")).forEach(System.out::println);
69     System.out.println("-----");
70
71     repo.searchByRecipeNameOrCookTimeOrderByCookTime("Biryani", "90mins").forEach(System.out::println);
72     System.out.println("-----");
73
74
75     repo.findAll().forEach(System.out::println);
76 }
77 }

```

Note:

The queries discussed above cover only the fundamental concepts. There are many more clauses, keywords, and advanced operations which might not be included here. Students are encouraged to explore beyond these notes and practice extensively to gain a deeper understanding.

One to One Mapping

User.java (Entity class)

```
Profile.java X User.java X
1 package in.springboot.entity;
2
3 import jakarta.persistence.Column;
4 import jakarta.persistence.Entity;
5 import jakarta.persistence.Id;
6 import jakarta.persistence.JoinColumn;
7 import jakarta.persistence.OneToOne;
8 import lombok.AllArgsConstructor;
9 import lombok.Data;
10 import lombok.NoArgsConstructor;
11
12 @Entity
13 @Data
14 @AllArgsConstructor
15 @NoArgsConstructor
16 public class User {
17
18     @Id
19     @Column(name = "uid")
20     private Integer id;
21
22     @Column(name = "username", nullable = false, length = 100)
23     private String username;
24
25     @Column(name = "email", unique = true, nullable = false, length = 150)
26     private String email;
27
28     @Column(name = "password", nullable = false)
29     private String password;
30
31     // One-to-One relation with Profile
32     @OneToOne
33     @JoinColumn(name="userFk")
34     private Profile profile;
35 }
36
```

UserRepo.java (Interface)

```
ProfileRepo.java UserRepo.java X
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.User;
6
7 public interface UserRepo extends JpaRepository<User, Integer> {
8
9 }
10
```


Profile.java (Entity class)

```
Profile.java X User.java
4
5 import jakarta.persistence.Column;
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.Table;
9 import lombok.AllArgsConstructor;
10 import lombok.Data;
11 import lombok.NoArgsConstructor;
12
13 @Entity
14 @Data
15 @AllArgsConstructor
16 @NoArgsConstructor
17 @Table(name = "Profile")
18 public class Profile {
19
20     @Id
21     @Column(name = "pid")
22     private Integer id;
23
24     @Column(name = "fname", nullable = false, length = 100)
25     private String fname;
26
27     @Column(name = "lname", nullable = false, length = 100)
28     private String lname;
29
30     @Column(name = "dob")
31     private LocalDate dob;
32
33     @Column(name = "gender", length = 10)
34     private String gender;
35
36     @Column(name = "phone", length = 15)
37     private String phone;
38
39     @Column(name = "address", length = 250)
40     private String address;
41 }
42
```

ProfileRepo.java (Interface)

```
ProfileRepo.java X UserRepo.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Profile;
6
7 public interface ProfileRepo extends JpaRepository<Profile, Integer> {
8
9 }
10
```

TestRunner.java (Class which implements CommandLineRunner)

```
TestRunner.java X
7 import in.springboot.entity.User;
8 import in.springboot.entity.Profile;
9 import in.springboot.repo.UserRepo;
10 import in.springboot.repo.ProfileRepo;
11
12 import java.time.LocalDate;
13
14 @Component
15 public class TestRunner implements CommandLineRunner {
16
17     @Autowired
18     private UserRepo userRepo;
19
20     @Autowired
21     private ProfileRepo profileRepo;
22
23     @Override
24     public void run(String... args) throws Exception {
25
26         // Create Profiles first
27         Profile p1 = new Profile(101, "Gagan", "Kulkarni", LocalDate.of(1995, 5, 20),
28             "Male", "9876543210", "123 Main Street");
29         Profile p2 = new Profile(102, "Bhargavi", "Sharma", LocalDate.of(1998, 8, 15),
30             "Female", "9876501234", "456 Park Avenue");
31
32         profileRepo.save(p1);
33         profileRepo.save(p2);
34
35         // Create Users and link Profiles
36         User u1 = new User(1, "gagan_k", "gagan12@gmail.com", "gagan_123", p1);
37         User u2 = new User(2, "bhargavi.sharma", "bhargavi.s@gmail.com", "bhargavi1212", p2);
38
39         userRepo.save(u1);
40         userRepo.save(u2);
41
42         System.out.println("Data inserted successfully!");
43     }
44 }
45
```

One to Many Mapping

Department.java(Entity class)

```
Department.java × Emp1.java DeptRepo.java EmpRepo.java TestRunner.java
1 package in.springboot.entity;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5 import java.util.Set;
6
7 @Entity
8 @Data
9 @AllArgsConstructor
10 @NoArgsConstructor
11 @Table(name="depttab")
12 public class Department {
13
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16
17     private Integer deptId;
18
19     @Column(nullable = false, length = 50)
20     private String deptName;
21
22     // One-to-Many
23     @OneToMany
24     @JoinColumn(name="deptFk")
25     private Set<Emp1> employees;
26 }
27
```

Emp1.java(Entity class)

```
Department.java Emp1.java × DeptRepo.java EmpRepo.java TestRunner.java
1 package in.springboot.entity;
2
3 import jakarta.persistence.*;
4 import lombok.*;
5
6 @Entity
7 @Data
8 @AllArgsConstructor
9 @NoArgsConstructor
10 public class Emp1 {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Integer empId;
15
16     private String empName;
17     private String designation;
18
19 }
20
```

DeptRepo.java(Interface)

```
Department.java  Emp1.java  DeptRepo.java X  EmpRepo.java  TestRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Department;
6
7 public interface DeptRepo extends JpaRepository<Department, Integer>{
8
9 }
10
```

EmpRepo.java(Interface)

```
Department.java  Emp1.java  DeptRepo.java  EmpRepo.java X  TestRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Emp1;
6
7 public interface EmpRepo extends JpaRepository<Emp1, Integer> {
8
9 }
10
```

TestRunner.java(class which implements CommandLineRunner)

```
Department.java  Emp1.java  DeptRepo.java  EmpRepo.java  TestRunner.java X
1 package in.springboot.runner;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.boot.CommandLineRunner;
5 import org.springframework.stereotype.Component;
6
7 import in.springboot.entity.Department;
8 import in.springboot.entity.Emp1;
9 import in.springboot.repo.DeptRepo;
10 import in.springboot.repo.EmpRepo;
11
12 import java.util.Arrays;
13 import java.util.HashSet;
14 import java.util.Set;
15
16 @Component
17 public class TestRunner implements CommandLineRunner {
18
19     @Autowired
20     private DeptRepo deptRepo;
21
22     @Autowired
23     private EmpRepo empRepo;
24
25     @Override
26     public void run(String... args) throws Exception {
27
28         // Create Employees (Emp1 objects)
29         Emp1 e1 = new Emp1(null, "Akash", "Developer");
30         Emp1 e2 = new Emp1(null, "Bharath", "Tester");
31         empRepo.saveAll(Arrays.asList(e1,e2));
32
33         Set<Emp1> emp = new HashSet<>();
34         emp.add(e1);
35         emp.add(e2);
36
37     }
38 }
```

```

36
37     // Create Department
38     Department d1 = new Department();
39     d1.setDeptName("Computer Science");
40
41     // Link employees to department
42     d1.setEmployees(emp);
43
44     // Save Department
45     deptRepo.save(d1);
46
47     System.out.println("Department with employees (Akash & Bharath) inserted successfully!");
48 }
49 }
50

```

Many to One Mapping

Movie.java(Entity class)

```

Movie.java × Ticket.java MovieRepository.java TicketRepository.java MovieRunner.java
1 package in.springboot.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 @Data
10 @AllArgsConstructor
11 @NoArgsConstructor
12 @Entity
13 public class Movie {
14
15     @Id
16     private Integer movieId;
17     private String movieName;
18     private String language;
19 }
20

```

Ticket.java(Entity class)

```
Movie.java Ticket.java X MovieRepository.java TicketRepository.java MovieRunner.java
1 package in.springboot.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import jakarta.persistence.JoinColumn;
6 import jakarta.persistence.ManyToOne;
7 import lombok.AllArgsConstructor;
8 import lombok.Data;
9 import lombok.NoArgsConstructor;
10
11 @Data
12 @Entity
13 @AllArgsConstructor
14 @NoArgsConstructor
15 public class Ticket {
16
17     @Id
18     private Integer ticketId;
19     private String seatNumber;
20     private double price;
21
22     @ManyToOne
23     @JoinColumn(name="movie_id")
24     private Movie movie;
25 }
26
```

MovieRepository.java(Interface)

```
Movie.java Ticket.java MovieRepository.java X TicketRepository.java MovieRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Movie;
6
7 public interface MovieRepository extends JpaRepository<Movie, Integer>{
8
9 }
10
```

TicketRepository.java(Interface)

```
Movie.java Ticket.java MovieRepository.java TicketRepository.java X MovieRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Ticket;
6
7 public interface TicketRepository extends JpaRepository<Ticket, Integer>
8 {
9
10 }
11
```

MovieRunner.java(class which implements CommandLineRunner)

```
Movie.java Ticket.java MovieRepository.java TicketRepository.java MovieRunner.java X
1 package in.springboot.runner;
2
3 import java.util.Arrays;
4
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 import in.springboot.entity.Movie;
10 import in.springboot.entity.Ticket;
11 import in.springboot.repo.MovieRepository;
12 import in.springboot.repo.TicketRepository;
13
14 @Component
15 public class MovieRunner implements CommandLineRunner{
16
17     @Autowired
18     MovieRepository movieRepo;
19
20     @Autowired
21     TicketRepository ticketRepo;
22
23     @Override
24     public void run(String... args) throws Exception {
25         // TODO Auto-generated method stub
26
27         Movie m1 = new Movie(100,"Harry Potter I", "English");
28         Movie m2 = new Movie(101,"Piku", "Hindhi");
29         Movie m3 = new Movie(102,"KGF", "Kannada");
30
31         movieRepo.saveAll(Arrays.asList(m1,m2,m3));
32
33         Ticket t1 = new Ticket(201, "L-10", 300.00, m1);
34         Ticket t2 = new Ticket(202, "L-11", 300.00, m1);
35         Ticket t3 = new Ticket(203, "L-12", 300.00, m1);
36         Ticket t4 = new Ticket(204, "M-7", 300.00, m2);
37         Ticket t5 = new Ticket(205, "F-9", 300.00, m3);
38         Ticket t6 = new Ticket(206, "E-12", 300.00, m2);
39         Ticket t7 = new Ticket(207, "E-1", 300.00, m3);
40         Ticket t8 = new Ticket(208, "E-2", 300.00, m3);
41
42         ticketRepo.saveAll(Arrays.asList(t1,t2,t3,t4,t5,t6,t7,t8));
43
44     }
45 }
46
47 }
```

Many to Many Mapping

Actor.java(Entity class)

```
Actor.java × Movie.java ActorRepository.java MovieRepository.java TestRunner.java
1 package in.springboot.entity;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.Id;
5 import lombok.AllArgsConstructor;
6 import lombok.Data;
7 import lombok.NoArgsConstructor;
8
9 @Data
10 @AllArgsConstructor
11 @NoArgsConstructor
12 @Entity
13 public class Actor {
14
15     @Id
16     private Integer id;
17     private String name;
18 }
19
20
```

Movie.java(Entity class)

```
Actor.java Movie.java × ActorRepository.java MovieRepository.java TestRunner.java
1 package in.springboot.entity;
2
3
4 import java.util.Set;
5
6 import jakarta.persistence.Entity;
7 import jakarta.persistence.Id;
8 import jakarta.persistence.JoinColumn;
9 import jakarta.persistence.JoinTable;
10 import jakarta.persistence.ManyToMany;
11 import lombok.AllArgsConstructor;
12 import lombok.Data;
13 import lombok.NoArgsConstructor;
14
15 @Data
16 @AllArgsConstructor
17 @NoArgsConstructor
18 @Entity
19 public class Movie {
20
21     @Id
22     private Integer id;
23     private String title;
24
25     @ManyToMany
26     @JoinTable(name="movieFk",
27         joinColumns = @JoinColumn(name="movieIdFk"),
28         inverseJoinColumns = @JoinColumn(name="actorIdFk"))
29     private Set<Actor> actors;
30 }
31
```

ActorRepository.java(Interface)


```
Actor.java  Movie.java  ActorRepository.java  MovieRepository.java  TestRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Actor;
6
7 public interface ActorRepository extends JpaRepository<Actor, Integer>{
8
9 }
10
```

MovieRepository.java(Interface)

```
Actor.java  Movie.java  ActorRepository.java  MovieRepository.java  TestRunner.java
1 package in.springboot.repo;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 import in.springboot.entity.Movie;
6
7 public interface MovieRepository extends JpaRepository<Movie, Integer>{
8
9 }
10
```

TestRunner.java(class which implements CommandLineRunner)

Actor.java Movie.java ActorRepository.java MovieRepository.java TestRunner.java ×

```
1 package in.springboot.runner;
2
3 import java.util.Arrays;
4 import java.util.Set;
5
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.boot.CommandLineRunner;
8 import org.springframework.stereotype.Component;
9
10 import in.springboot.entity.Actor;
11 import in.springboot.entity.Movie;
12 import in.springboot.repo.ActorRepository;
13 import in.springboot.repo.MovieRepository;
14 import jakarta.transaction.Transactional;
15
16 @Component
17 public class TestRunner implements CommandLineRunner{
18
19     @Autowired
20     MovieRepository mrepo;
21
22     @Autowired
23     ActorRepository arepo;
24
25     @Transactional
26     @Override
27     public void run(String... args) throws Exception {
28         // TODO Auto-generated method stub
29
30         Actor a1 = new Actor(100, "Amithabh bacchan");
31         Actor a2 = new Actor(101, "Irrfan Khan");
32         Actor a3 = new Actor(102, "Deepika Padukone");
33         Actor a4 = new Actor(103, "Ranveer Singh");
34
35
36         arepo.saveAll(Arrays.asList(a1,a2,a3,a4));
37
38
39         Movie m1 = new Movie(222, "Ramleela", Set.of(a3,a4));
40         Movie m2 = new Movie(223, "Piku", Set.of(a1,a2,a3));
41
42         mrepo.saveAll(Arrays.asList(m1,m2));
43
44         arepo.findAll().forEach(System.out::println);
45         mrepo.findAll().forEach(System.out::println);
46     }
47 }
48 }
49
```